

Answer to all questions.

Allowed Time: 90 Minutes

Instructions:

- Answer all question.
- Write all code in plain JavaScript using correct syntax and indentation.
- Use meaningful variable and function names.
- Marks will be awarded for correct logic even if minor syntax slips occur, provided the intention is clear.

Question: Library Management System

Scenario:

CityLib Library wants a simplified system to manage its book lending process. The library deals with two types of members: **Regular Members** and **Premium Members**. Every member has a member ID, name, and a count of books currently borrowed. The library has specific rules:

- Regular members can borrow a maximum of 3 books at a time and are charged Rs. 10 per day for any overdue book.
- Premium members can borrow up to 10 books at a time and are charged a discounted overdue fee of Rs. 5 per day.
- The number of books borrowed must never be modified directly from outside the class — it should only change through controlled methods (`borrowBook()` / `returnBook()`).
- All member types must support a common way to calculate overdue fines, even though the fee and borrowing limit differ.

Tasks:

- a) Define a class `LibraryMember` with common properties (member ID, name, `booksBorrowed` count) as **encapsulated (private/protected)** fields with getter methods, and a method `calculateFine(overdueDays: number): number` with a basic default implementation. (6 marks)
- b) Create classes `RegularMember` and `PremiumMember` that **inherit** from `LibraryMember` and **override** `calculateFine()` to apply their own daily rate. (6 marks)
- c) Implement `borrowBook()` and `returnBook()` methods in the base class using **encapsulation**, ensuring `RegularMember` enforces a 3-book limit and `PremiumMember` enforces a 10-book limit (override where necessary, and prevent borrowing beyond the limit or returning when no books are borrowed). (6 marks)
- d) Write a function `generateOverdueReport(members: LibraryMember[], overdueDays: number)` that loops through an array of mixed member **objects** and calls `calculateFine()` on each, printing the member's name and fine amount, demonstrating **polymorphism**. (4 marks)
- e) In your main program, create at least one `RegularMember` object and one `PremiumMember` object, borrow books up to (and one attempt beyond) their limit, return a book, and call `generateOverdueReport()` for both. (3 marks)